

Recovering a Time-Varying Transmission Rate with Simulation-Based Inference

Yixin Ma

Summer 2026

Abstract

Estimating a time-varying transmission rate from reported case data is challenging because epidemic transmission dynamics are stochastic and only partially observed. Within a partially observed Markov process framework, this report compares two representations of the transmission rate $B(t)$: a latent Gamma-noise process that allows $B(t)$ to vary over time and a constant- B formulation. Both models are fitted using the same likelihood-based inference framework. Particle filtering is used to approximate the likelihood and filtering distribution, while iterated filtering (IF2) is used to estimate the static parameters.

The comparison is evaluated using a simulation benchmark in which the prescribed deterministic transmission rate changes from $B(t) = 4$ before week 5 to $B(t) = 2$ from week 5 onward. Across 200 paired simulated data sets satisfying the acceptance criterion, the Gamma-noise fitting model is compared with a stochastic epidemic fitting model having a constant transmission rate. Recovery accuracy is measured using the root mean squared error (RMSE) between one ancestry-preserving sampled latent trajectory from the selected Gamma fit and the prescribed transmission-rate trajectory; the constant-model estimate is repeated over time. Under this benchmark, mean RMSE was 0.653 for Gamma-noise and 1.199 for constant- B , with lower Gamma-noise RMSE in 195 of 200 paired data sets.

Contents

1	Introduction	3
2	Model	4
2.1	Three POMP models	4
2.2	Stochastic SIR process	5
2.3	Observation model	5
2.4	Transmission-rate models	6
3	Inference and evaluation	7
3.1	Likelihood and maximum-likelihood estimation	7
3.2	Particle filtering	7
3.3	IF2 estimation	7
3.4	Transmission-rate recovery	8
4	Simulation design	9
5	Results	10
5.1	Primary recovery result	10
5.2	Secondary recovery summaries	10

6	Discussion	11
6.1	Why the Gamma-noise model performed better	11
6.2	What the result means	11
6.3	Limitations and next step	12
A	Developmental experiments	13
B	Supporting diagnostics	13
B.1	Recovery-error distributions	13
B.2	Post-fitting likelihood diagnostic	13
B.3	Available convergence diagnostics	13
C	Inference settings and R code listings	15
C.1	Inference settings	15
C.2	Supplied R files	15
C.3	Experiment configuration	16
C.4	Data-generating and fitting POMPs	17
C.5	IF2 fitting and post-fit filtering	20
C.6	Paired recovery calculations	24
D	Reproducibility information	26
D.1	Retained inputs and outputs	26
D.2	Trajectory and likelihood semantics	27
D.3	Exact-reproduction boundary	27

1 Introduction

Infectious disease outbreaks are commonly observed through univariate or multivariate time series of reported cases, such as the number of newly reported cases aggregated over a specified time interval. These data record when and how many cases were reported, but they do not directly reveal the underlying process by which infection spreads through a population. Mechanistic epidemic models connect the observed cases to changes in susceptible, infectious, and recovered populations by explicitly representing the dynamics of disease transmission Bretó et al. (2009). Partially observed Markov process (POMP) models provide a general framework for such models by separating two components: a latent Markov process describing the transmission dynamics and an observation process linking the latent states to the reported cases King et al. (2016).

Statistical inference for stochastic POMP models is generally difficult. Even when the static parameters and initial conditions are fixed, a stochastic model can generate many different latent epidemic trajectories. Evaluating the likelihood of the observed data requires integrating over these unobserved epidemic states and transmission histories, and the resulting likelihood therefore does not usually have a simple closed-form expression. Many statistical methods have been proposed for inference on POMP models Bretó et al. (2009); King et al. (2016). These methods make it possible to use reported cases to infer epidemic processes and parameters that cannot be observed directly.

Within this framework, this report focuses on how to represent and infer a transmission rate that varies over time. The constant- B POMP model assumes that the transmission rate remains fixed throughout the epidemic. In practice, the transmission rate may change with contact behavior, public-health interventions, and other conditions, while the transmission rate itself cannot be observed directly from reported cases Bouman et al. (2024). This report therefore incorporates Gamma noise into the process model and represents the transmission rate as a positive latent stochastic process Bretó et al. (2009). The Gamma-noise POMP model is then compared with the constant- B POMP model to assess their ability to infer a time-varying transmission rate from reported cases.

This study is a simulation-based model comparison and a transmission-rate recovery study. Its central question is whether, under a prescribed piecewise truth in which the transmission rate decreases from 4 to 2 beginning at week 5, the Gamma-noise POMP model can recover the known time-varying transmission rate more accurately than the constant- B POMP model. The experimental design contains three distinct POMP models. A data-generating POMP combines the prescribed transmission-rate path with a stochastic SIR transmission process and a negative-binomial observation model to generate reported cases. The same reported cases are then supplied to two fitting POMP models. The Gamma-noise POMP model treats the transmission rate as a time-varying latent stochastic process, whereas the constant- B POMP model estimates a single transmission-rate parameter for the entire epidemic. Neither fitting model is given the true transmission-rate path or the location of its change point.

For a fixed set of parameter values, the particle filter is used to estimate the likelihood of the observed data and obtain filtering summaries of the transmission rate and other latent states Gordon et al. (1993); King et al. (2016). Iterated filtering (IF2) uses this particle-filtering framework to search the static parameter space of each fitting model Ionides et al. (2015). These algorithms support statistical inference for the model comparison; they are not components of the epidemic models themselves. The inferential performance of the two models is evaluated primarily by comparing the estimated transmission rates with the known truth using recovery error and root mean squared error (RMSE).

This report investigates the ability of the Gamma-noise POMP model to infer a time-varying transmission rate and compares its performance with that of the constant- B POMP model in a controlled simulation study. The results show that, when the prescribed transmission rate decreases

from 4 to 2, the Gamma-noise POMP model recovers the transmission-rate path more accurately and produces a lower RMSE. Section 2 defines the stochastic epidemic process, the observation process, and the two transmission models. Section 3 introduces the particle filter and IF2 used for inference, together with the measures used to evaluate recovery performance. Sections 4–6 present the simulation design, the results, and their interpretation.

2 Model

2.1 Three POMP models

Experiment 4 uses one data-generating POMP and two fitting POMP (Figure 1). All three models contain the same stochastic epidemic transitions and negative-binomial observation process. They differ only in the source of the transmission rate used by the epidemic process. The data-generating model uses a prescribed deterministic path $B_{\text{true}}(t)$; the Gamma-noise fitting model includes $B(t)$ as a positive latent stochastic state; and the constant- B fitting model uses one positive static parameter B_{const} .

This separation makes the comparison interpretable. The generator defines the target against which recovery is measured, whereas the two fitting models define alternative explanations of the same observations. Calling the prescribed path deterministic refers only to its transmission-rate input: the epidemic trajectory and the reported cases generated from it are random. In the same way, calling the comparator constant refers only to its transmission rate; its epidemic and observation processes remain stochastic.

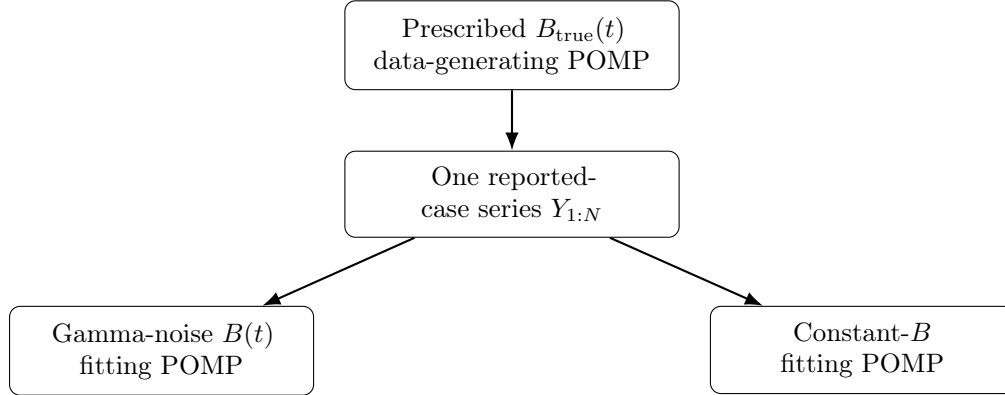


Figure 1: Three-model design. Each simulated case series is supplied unchanged to both fitting POMP.

Immediately before measurement at observation time t_n , the Gamma-noise state is

$$X_n^{(G)} = \{S(t_n), I(t_n), R(t_n), H(t_n), B(t_n)\}.$$

The data-generating and constant- B states omit the dynamic transmission component:

$$X_n^{(D)} = X_n^{(C)} = \{S(t_n), I(t_n), R(t_n), H(t_n)\}.$$

The variable H is an incidence accumulator. It records infections since the previous observation and is reset after the corresponding measurement is generated.

2.2 Stochastic SIR process

Let S_k , I_k , and R_k be the susceptible, infectious, and recovered counts at Euler time t_k , with step length Δt . Write B_k^{use} for the transmission value supplied by the relevant model. The one-step infection and recovery probabilities are

$$p_{SI,k} = 1 - \exp(-B_k^{\text{use}} I_k \Delta t / N), \quad p_{IR,k} = 1 - \exp(-\mu_{IR} \Delta t). \quad (1)$$

Conditional on the current state, the event counts are

$$\Delta N_{SI,k} \sim \text{Binomial}(S_k, p_{SI,k}), \quad \Delta N_{IR,k} \sim \text{Binomial}(I_k, p_{IR,k}). \quad (2)$$

The factor $B_k^{\text{use}} I_k / N$ is the per-susceptible infection hazard. The exponential conversion gives the probability of infection over the finite step Δt , while the recovery probability is derived from the constant recovery hazard μ_{IR} . This construction keeps both probabilities in $[0, 1]$ and lets the same transition equations accept any of the three transmission inputs. The state then changes according to

$$S_{k+1} = S_k - \Delta N_{SI,k}, \quad (3)$$

$$I_{k+1} = I_k + \Delta N_{SI,k} - \Delta N_{IR,k}, \quad (4)$$

$$R_{k+1} = R_k + \Delta N_{IR,k}, \quad (5)$$

$$H_{k+1} = H_k + \Delta N_{SI,k}. \quad (6)$$

Thus $S_k + I_k + R_k = N$, while H_k records events rather than people and is excluded from the population total. The binomial draws make the epidemic process stochastic even when B_k^{use} is deterministic or constant.

The accumulator bridges the process and observation time scales. The SIR state is updated every Euler step, but cases are observed only once per day. Summing $\Delta N_{SI,k}$ into H preserves all infections between two measurements; resetting H after measurement prevents the same infection from contributing to more than one reported count.

2.3 Observation model

Let H_n be the infections accumulated during $(t_{n-1}, t_n]$, immediately before reset, and let Y_n be the reported case count. Reported cases depend on incidence H_n , not on the number infectious at one instant. The model is

$$Y_n \mid H_n \sim \text{NegBin}(\text{mean} = \rho H_n, \text{size} = k), \quad (7)$$

with

$$\mathbb{E}(Y_n \mid H_n) = \rho H_n, \quad \text{Var}(Y_n \mid H_n) = \rho H_n + \frac{(\rho H_n)^2}{k}.$$

Here ρ controls the expected reported fraction and k controls overdispersion around that conditional mean. The observation model does not feed reports back into the epidemic: Y_n is generated from H_n , after which the latent SIR process continues from its current state. This negative-binomial variation is observation noise conditional on latent incidence. It is distinct from the binomial demographic variation in the SIR transitions.

2.4 Transmission-rate models

Gamma-noise model. The Gamma-noise fitting POMP initializes $B(0) = B_0 > 0$. At each Euler step it draws

$$B_{k+1} \mid B_k \sim \text{Gamma} \left(\text{shape} = \frac{1}{\sigma_B^2 \Delta t}, \text{scale} = B_k \sigma_B^2 \Delta t \right), \quad (8)$$

and uses the updated value B_{k+1} immediately in the infection probability. The two static transmission parameters are B_0 and σ_B ; the sequence $B(t)$ remains latent.

For the shape–scale parameterization in Equation 8,

$$\mathbb{E}(B_{k+1} \mid B_k) = B_k, \quad (9)$$

$$\text{Var}(B_{k+1} \mid B_k) = B_k^2 \sigma_B^2 \Delta t. \quad (10)$$

The transition therefore has zero conditional drift and one-step conditional coefficient of variation $\sigma_B \sqrt{\Delta t}$. Larger σ_B permits greater relative movement between adjacent process times. Because the Gamma distribution has positive support, $B(t)$ remains positive. When time is measured in weeks, $B(t)$ has units week^{-1} and σ_B has units $\text{week}^{-1/2}$.

Zero conditional drift does not force a realized path to remain flat. A draw can move above or below the current value, and successive innovations can accumulate into a sustained change. Conversely, the transition contains no preferred change point, downward trend, or mean-reversion level. Evidence for a decline must therefore enter through the observed cases during filtering. The role of σ_B is specifically to regulate local relative process variation; it is not a reporting-dispersion parameter and it is not an IF2 search perturbation.

Prescribed data-generating rate. The data-generating POMP sets

$$B_k^{\text{use}} = B_{\text{true}}(t_k).$$

This path is fixed by the simulation design rather than drawn from Equation 8. The generated epidemic and reports are still random because their binomial transitions and negative-binomial measurements remain stochastic. Section 4 gives the prescribed path and parameter values.

Constant- B model. The constant fitting POMP estimates one positive parameter B_{const} and sets

$$B_k^{\text{use}} = B_{\text{const}}$$

throughout the epidemic. It has no latent transmission-rate state, but it retains the same stochastic SIR and observation processes as the other two POMPs.

The scientific dependence common to all three models is

$$B(t) \longrightarrow \text{infection hazard} \longrightarrow \text{new infections} \longrightarrow H_n \longrightarrow Y_n.$$

The Gamma-noise model adds stochastic variation at the first link. Particle filtering and IF2, introduced next, are computational tools for inference and do not add scientific transitions to this chain.

Five sources of variation should therefore be kept separate. Gamma process noise moves the latent transmission rate only in the Gamma-noise fitting POMP; binomial demographic noise moves the epidemic counts in all three POMPs; and negative-binomial observation noise moves reported cases around latent incidence. Particle-filter Monte Carlo error is numerical approximation error from using finitely many particles, while IF2 perturbations are temporary optimization devices. The last two affect inference but are not components of the scientific epidemic model.

3 Inference and evaluation

3.1 Likelihood and maximum-likelihood estimation

The observed case series does not reveal the epidemic states or, in the Gamma-noise model, the transmission-rate path. The observed-data likelihood therefore averages over all latent state histories compatible with a fixed parameter vector. That integral is not available analytically for these nonlinear stochastic POMP models. The inferential target is the maximum likelihood estimate

$$\hat{\theta} = \arg \max_{\theta} \ell(\theta; y_{1:N}). \quad (11)$$

The model-specific static transmission parameters are

$$\theta_G = (B_0, \sigma_B), \quad \theta_C = (B_{\text{const}}).$$

The nuisance parameters μ_{IR}, N, ρ, k and the initial epidemic state are fixed in this benchmark. In particular, the latent sequence $B(t)$ is not optimized as an unrelated parameter at every time point; it is integrated over according to the Gamma transition in Equation 8.

3.2 Particle filtering

For a fixed parameter vector, a particle filter propagates many possible latent states through the process model, weights them according to each new reported count, and resamples states that better explain the observations. These weighted particles approximate both the observed-data likelihood and the filtering distribution of the latent state given reports through the current observation time (Gordon et al., 1993; King et al., 2016). The analysis uses the implementation in the `pomp` package (King et al., 2025).

At an observation time, weighting favors particles whose accumulated incidence makes the reported count plausible; resampling then concentrates subsequent computation on those histories without collapsing the scientific model to a single path. Because only finitely many particles are used, two independent filters at the same parameter vector need not return identical likelihood estimates or sampled latent histories. This particle-filter Monte Carlo variation is reduced by increasing the particle population and is separate from every biological or reporting source of randomness in Section 2.

3.3 IF2 estimation

Iterated filtering (IF2) connects particle filtering to the optimization in Equation 11 (Ionides et al., 2015). IF2 repeatedly applies a particle filter to an extended parameter–state system. Artificial perturbations allow the static-parameter particles to explore the likelihood surface, while observation weighting and resampling favor parameter–state combinations that better explain the data. A cooling schedule reduces the perturbation scale across iterations, producing an approximate maximum likelihood estimate. Multiple starting points broaden the search but do not guarantee that the global maximum has been found.

In the implemented search, each starting point underwent a fixed sequence of 600 IF2 iterations. The decreasing perturbations turn the early iterations into relatively broad exploration and the later iterations into a more local search. Candidate endpoints were not ranked using the final IF2 trace value alone: five independent ordinary particle filters reevaluated each endpoint after perturbations

had been removed. If ℓ_{dsj} is the log-likelihood estimate for data set d , start s , and evaluation j , the ranking summary was

$$\tilde{\ell}_{ds} = \log \left\{ \frac{1}{5} \sum_{j=1}^5 \exp(\ell_{dsj}) \right\}, \quad (12)$$

computed numerically with `pomp::logmeanexp`. Thus the averaging occurs on the likelihood scale. It is neither an arithmetic mean of log likelihoods nor a mean of the latent transmission trajectories. The endpoint with the largest finite $\tilde{\ell}_{ds}$ was selected as the best fit. This separates parameter search from the final comparison as far as the finite computation allows.

IF2 perturbations are algorithmic and disappear from the focal model after optimization. They are distinct from Gamma process noise, which governs the scientific evolution of latent $B(t)$ at the fitted value of σ_B . Appendix C records the starting values, particle counts, perturbation scales, cooling, and selection details.

3.4 Transmission-rate recovery

For paired data set d , the selected Gamma parameters were passed to one final 50,000-particle filter with trajectory ancestry retained. The `filter_traj()` function then extracted one internally consistent latent history,

$$\hat{B}_d^{(G)}(t_{0:N}) = B_d^*(t_{0:N}; \hat{\theta}_{G,d}), \quad (13)$$

where the superscript $*$ denotes the sampled history rather than a posterior or filtering mean. Because its ancestry is traced through the final particle population, this is a finite-particle, plug-in-parameter approximation to a smoothing trajectory conditional on the complete reported series. It is not an exact posterior draw, an uncertainty interval, or a separately optimized value of B at each time. The value at t_0 is retained only for display; all recovery metrics use the 70 observation times $t_1, \dots, t_N$.

The constant-model estimator is the selected static value repeated over those same observation times,

$$\hat{B}_d^{(C)}(t_n) = \hat{B}_{\text{const},d}.$$

Both models are therefore represented by one observation-time trajectory before comparison, even though the constant trajectory necessarily repeats a single value. No filtering mean enters the Experiment 4 recovery metrics. For model $m \in \{G, C\}$, the primary task-level outcome is

$$\text{RMSE}_d^{(m)} = \sqrt{\frac{1}{N_{\text{obs}}} \sum_{n=1}^{N_{\text{obs}}} \left[\hat{B}_d^{(m)}(t_n) - B_{\text{true}}(t_n) \right]^2}. \quad (14)$$

This definition compares like observation times within each paired data set. Each data set contributes one RMSE per model and therefore one within-pair difference. Averaging these task-level RMSE values gives equal weight to the 200 accepted epidemics, rather than allowing a data set with more cases to dominate the summary. Squaring before averaging also prevents early underestimation and late overestimation from canceling.

Secondary summaries describe signed recovery error. The task-level mean error averages $\hat{B}_d^{(m)}(t_n) - B_{\text{true}}(t_n)$ over all observation times; its absolute value measures overall displacement without allowing the final sign to determine magnitude. Segment-specific signed errors average the same quantity before and after week 5. Post-fitting particle-filter likelihoods provide an additional same-data fitting diagnostic, but they are neither penalized nor evaluated on held-out observations and are not treated as a formal model-selection test.

4 Simulation design

Experiment 4 is a paired recovery comparison. For each task, the data-generating POMP simulates an epidemic and reported cases once. The resulting observation times and case counts are saved and supplied unchanged to the Gamma-noise and constant- B fitting POMP. Pairing removes between-data-set variation from the model contrast: both fits must explain the same realized epidemic reports.

Within a task, the deterministic transmission path drives a new stochastic SIR realization, the accumulator records infections between daily observations, and the negative-binomial model produces one case count at each observation time. Only these observation times and counts enter the two fitting workflows. The unobserved generating states are retained for evaluation but are not supplied to IF2 or to the final filters.

The data-generating transmission path is

$$B_{\text{true}}(t) = \begin{cases} 4, & 0 \leq t < 5, \\ 2, & 5 \leq t \leq 10, \end{cases} \quad t \text{ in weeks.} \quad (15)$$

This path is deterministic. Neither fitting model receives $B_{\text{true}}(t)$, the values in Equation 15, or the week-5 change point. The principal settings are summarized in Table 1.

The 70 daily observations span the ten-week horizon; the scheduled observation at week 5 is evaluated under the post-change rate. The Euler grid is finer than the observation grid, with approximately four process updates between successive daily measurements. Fixing the recovery, reporting, population, and initial-state quantities at their generating values makes this a focused transmission-recovery experiment: it avoids adding nuisance-parameter confounding to the primary comparison.

Table 1: Experiment 4 simulation settings.

Setting	Value
Paired accepted data sets	200
Acceptance criterion	$\max(H) > 20$
Duration	10 weeks
Observation schedule	70 daily observations, 1/7, ..., 10 weeks
Euler step	$\Delta t = 1/30$ week
Prescribed transmission path	4 before week 5; 2 from week 5 onward
Population size	$N = 10000$
Recovery rate	$\mu_{IR} = 3 \text{ week}^{-1}$
Reporting fraction	$\rho = 0.5$
Negative-binomial size	$k = 10$
Initial state	$S(0) = 9990, I(0) = 10, R(0) = H(0) = 0$

The acceptance rule conditions the comparison on simulated outbreaks with more than 20 accumulated infections in at least one observation interval. Consequently, the reported recovery results describe accepted outbreaks under these settings rather than all trajectories that the data-generating POMP could produce. Experiments 1–3 were developmental steps used to build and calibrate the workflow; only Experiment 4 supplies the main scientific evidence. Their roles are summarized in Appendix A.

5 Results

5.1 Primary recovery result

The Gamma-noise model recovered the prescribed transmission path more accurately than the constant- B model on the primary outcome. Mean task-level RMSE was 0.653 for Gamma-noise and 1.199 for constant- B . Gamma-noise had lower RMSE in 195 of 200 paired data sets (97.5%). The task-by-task comparison in Figure 2 shows that the advantage was widespread rather than being driven by a small subset of simulated epidemics.

Relative to the constant-model mean, the difference corresponds to an approximately 45.5% reduction in mean RMSE. The five reversals of the paired ordering are visible above the equality line and prevent interpreting the result as uniform across all accepted realizations. The scatter also shows variation in absolute recovery difficulty, as expected from stochastic epidemic, reporting, and sampled-trajectory variation.

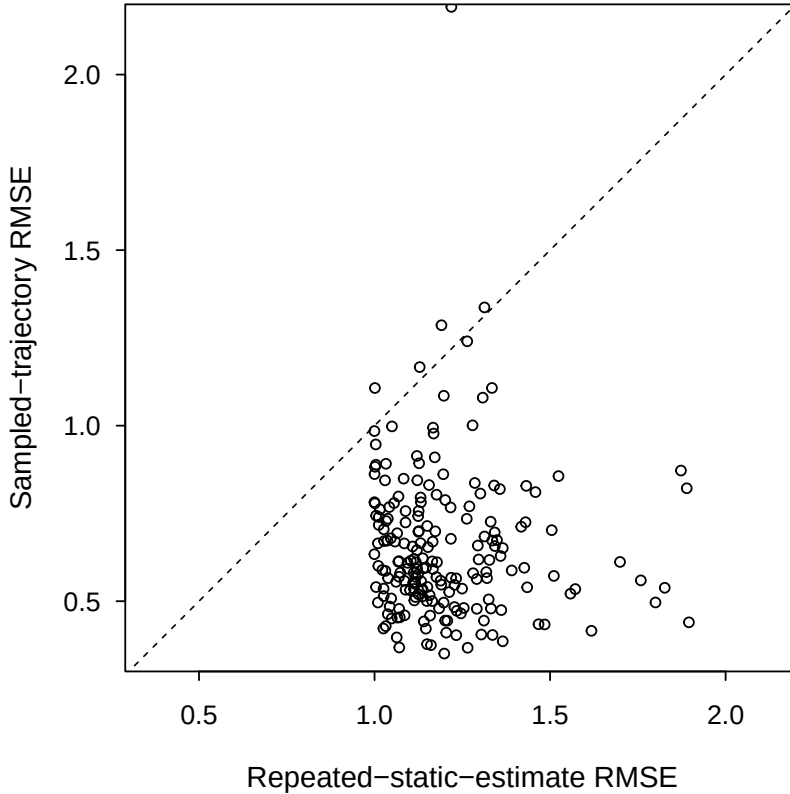


Figure 2: Paired RMSE calculated from the sampled Gamma trajectory and repeated constant estimate for the same 200 accepted data sets. Points below the diagonal favor Gamma-noise.

Table 2 places the primary result beside the secondary signed-error summaries.

5.2 Secondary recovery summaries

The mean absolute task-level mean error was 0.229 for Gamma-noise and 0.606 for constant- B . Gamma-noise had the smaller value in 177 of 200 data sets. The segment-specific errors explain the main weakness of a single static rate. Before week 5, mean signed error was -0.086 for Gamma-noise

Table 2: Mean task-level recovery summaries across 200 paired data sets.

Quantity	Gamma-noise	Constant- B
RMSE	0.653	1.199
Absolute task-level mean error	0.229	0.606
Signed mean error, $t < 5$	-0.086	-0.424
Signed mean error, $t \geq 5$	0.008	1.576

and -0.424 for constant- B . From week 5 onward, the corresponding errors were 0.008 and 1.576. The large positive constant- B error after the change indicates that one fitted rate remained well above the prescribed value of 2.

The signed errors also show why the primary RMSE and the absolute mean error answer different questions. A constant estimate between 2 and 4 can be too low in the first regime and too high in the second; averaging those signed errors permits partial cancellation. RMSE retains both discrepancies, while the segment summaries reveal their direction. Gamma-noise reduced both the post-change overshoot and the overall displacement, although it did not produce zero mean error in either segment.

Post-fitting likelihoods gave consistent supporting evidence: mean particle-filter log likelihood was -191.08 for Gamma-noise and -209.23 for constant- B , and the saved paired difference favored Gamma-noise in 199 of 200 tasks. This same-data, unpenalized comparison is a descriptive fitting diagnostic rather than a formal model-selection test. The error-distribution and likelihood-difference figures are retained in Appendix B.

These results summarize one sampled Gamma trajectory per accepted outbreak under the specified benchmark. They do not compare filtering means, uncertainty intervals, repeated trajectory draws, or performance under other transmission paths.

6 Discussion

6.1 Why the Gamma-noise model performed better

The prescribed epidemic contains two transmission regimes, but the constant- B model must explain both with one compromise value. That restriction produces a fitted rate that is too low before the change and much too high afterward. The Gamma-noise model can instead shift probability toward lower latent $B(t)$ paths as post-change reports arrive. This adaptation explains both its lower overall RMSE and its much smaller post-change signed error.

The Gamma transition does not encode the true week-5 boundary or a deterministic downward trend. Its conditional mean equals the current rate, so the fitted decrease emerges through conditioning on the observations: particle paths that better explain the later reports receive more weight. The comparison therefore shows the value of allowing transmission to change; it does not show that a Gamma process generated the prescribed discontinuity.

6.2 What the result means

The lower RMSE in 195 of 200 paired data sets shows that a positive latent-rate model usually recovered this imposed change more accurately than a static-rate model when both were fitted to the same noisy case series. Because all three POMP models retain the same stochastic SIR and observation

components, the contrast is attributable to how the fitting models represent transmission rather than to one model being stochastic and the other deterministic.

The secondary summaries support the same interpretation while measuring different features of error. Absolute task-level mean error can benefit from cancellation between early negative and later positive errors, so it need not rank models identically to RMSE in every data set. The post-fitting likelihood pattern shows that the more flexible model usually assigned higher likelihood to the fitted observations, but it does not account for model complexity or out-of-sample performance.

6.3 Limitations and next step

Three limitations define the scope of the conclusion:

- *Benchmark scope.* The experiment studies one abrupt path over ten weeks and conditions on outbreaks satisfying $\max(H) > 20$. Other change sizes, timings, smooth trajectories, or weaker outbreaks may yield different recovery behavior.
- *Fixed nuisance parameters.* The recovery rate, reporting model, population size, and initial state are fixed at their generating values. Applied analyses must confront confounding between these quantities and transmission, especially when a flexible σ_B can absorb other misspecification.
- *Finite optimization.* IF2 uses a finite particle population, cooling schedule, and set of starting points. The Gamma model received nine starts and the constant model six, so the experiment does not establish equal computational budgets or guarantee global maximization.
- *Single sampled trajectory.* Recovery metrics use one ancestry-preserving latent path from the final filter for each Gamma fit. They therefore include finite-particle and path-sampling variation and should not be interpreted as posterior-mean error or as a summary of trajectory uncertainty. Repeated trajectory extraction would be needed to quantify that additional uncertainty.

A useful next comparison would replace local Gamma innovations with a smooth basis representation,

$$\log B(t) = \sum_{q=1}^Q c_q \phi_q(t),$$

where $\phi_q(t)$ are B-spline basis functions (Bouman et al., 2024). Such a model would trade local stochastic adaptation for a globally smooth trajectory whose flexibility depends on the number and placement of knots and on regularization. Because smoothing can blur an abrupt change, the basis dimension and penalty would need to be prespecified and evaluated in a new simulation study. The next scientific question is therefore whether Gamma noise or a controlled smooth-basis model recovers a wider range of plausible transmission paths more reliably.

A Developmental experiments

Experiments 1–3 developed the modeling and computational workflow but do not contribute numerical evidence to the main comparison.

Table 3: Developmental progression to Experiment 4.

Experiment	Purpose	Why developmental	Link to Experiment 4
1	Construct and illustrate the Gamma-noise model under constant and piecewise transmission scenarios.	Each scenario used one simulated epidemic and primarily examined model construction and starting-value sensitivity.	Established the basic latent-rate recovery workflow.
2	Exercise a high-particle, multi-start Gamma-noise fitting workflow.	The workflow was applied to one fixed simulated data set.	Established the repeated-fitting workflow.
3	Evaluate Gamma-noise recovery across 200 tasks with $N_{\text{mif}} = 100$.	Its lower-iteration analysis was superseded for final numerical claims.	Motivated the $N_{\text{mif}} = 600$ paired comparison.
4	Compare Gamma-noise and constant- B recovery on shared data.	Main experiment rather than developmental evidence.	Supplies all numerical results in the main text.

B Supporting diagnostics

B.1 Recovery-error distributions

Figure 3 shows the distribution of task-level signed mean errors underlying the secondary summaries in Table 2. It is retained as supporting evidence rather than as a second presentation of the primary RMSE result.

B.2 Post-fitting likelihood diagnostic

Figure 4 displays the paired same-data particle-filter likelihood differences. Positive values favor Gamma-noise numerically, but the diagnostic is unpenalized and is not based on held-out observations.

B.3 Available convergence diagnostics

Complete IF2 traces were saved for tasks 1, 50, 100, 150, and 200. The files

- `figures/experiment4/convergence/gamma_convergence_diagnostics.pdf`,
- `figures/experiment4/convergence/constant_B_convergence_diagnostics.pdf`, and
- `figures/experiment4/convergence/convergence_tail_summary.csv`

contain the all-start trace panels and final-100-iteration slope summaries. They are preserved with the project but are not embedded here because the workflow defined no numerical pass/fail threshold. These selected-task traces provide qualitative checks on the finite IF2 runs rather than evidence of global convergence for all 200 data sets.

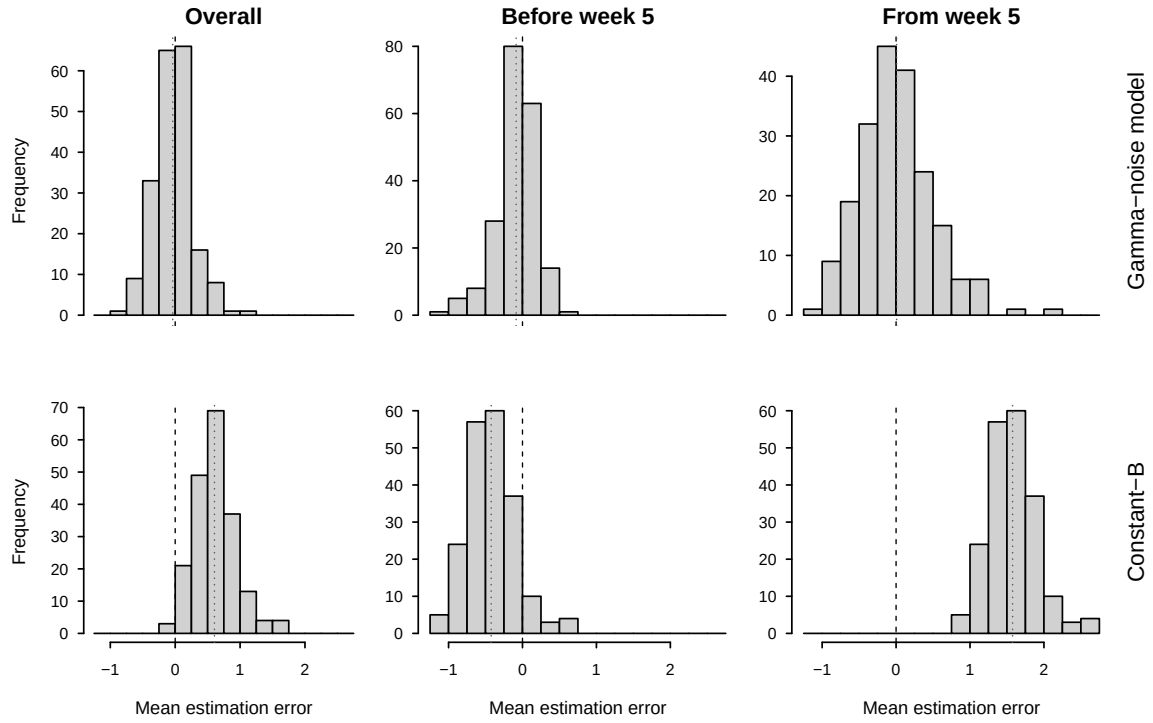


Figure 3: Task-level signed mean errors across 200 paired data sets. Dashed lines mark zero and dotted lines mark panel means.

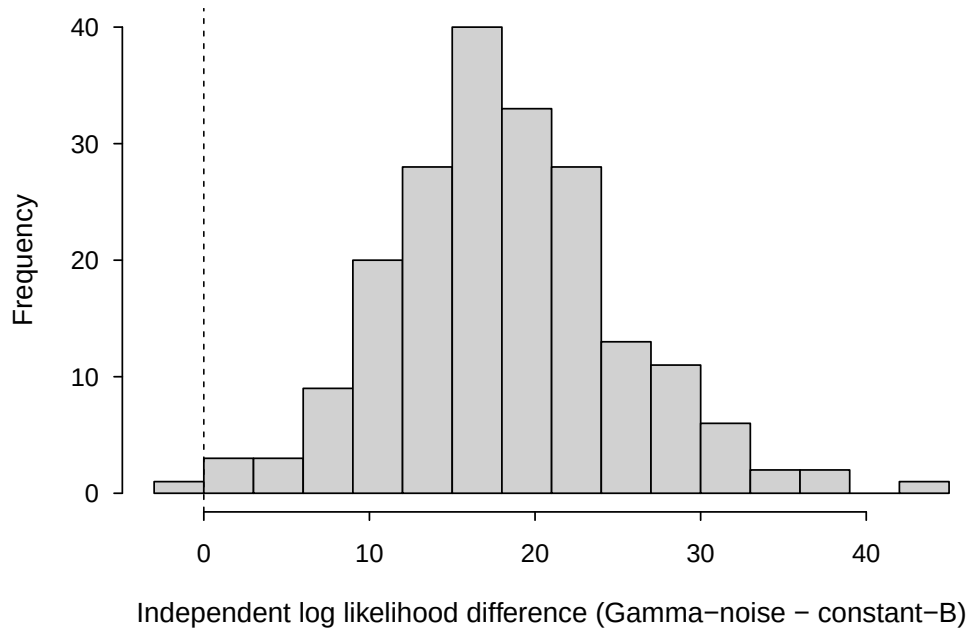


Figure 4: Gamma-noise minus constant- B post-fitting log-likelihood differences for 200 paired data sets.

C Inference settings and R code listings

C.1 Inference settings

Table 4: Model-specific IF2 search settings.

Model	Starting values	IF2 perturbation scale
Gamma-noise	$B_0 \in \{2, 4, 6\}$, $\sigma_B \in \{0.10, 0.30, 0.45\}$	$B_0 : \text{ivp}(0.20)$; $\sigma_B : 0.05$
Constant- B	$B_{\text{const}} \in \{1, 2, 3, 4, 5, 6\}$	$B_{\text{const}} : 0.05$

Each start used 600 IF2 iterations, 5,000 particles per iteration, and geometric cooling with half-life fraction 0.5. Each fitted start was then evaluated with five independent 50,000-particle filters. Finite likelihood estimates were combined with `pomp::logmeanexp`, and the start with the largest summary was selected. One final 50,000-particle filter retained trajectory ancestry; `filter_traj()` then extracted the sampled Gamma path used in both the recovery metrics and trajectory figures. Because the Gamma model used nine starts and the constant model six, the total search budgets were unequal.

C.2 Supplied R files

The executable source is retained under `code/experiment4_source/`. The index below locates each workflow stage; longer exact excerpts from the central scripts follow the table.

Table 5: Experiment 4 source-file index.

File	Purpose
<code>experiment_config.R</code>	Scientific parameters, simulation design, IF2 settings, and starting grids.
<code>model_components.R</code>	Data-generating, Gamma-noise, and constant- B POMP definitions.
<code>01_generate_shared_data_task.R</code>	Generate and save one accepted data set for both fitting models.
<code>02_run_gamma_task.R</code>	Multi-start IF2, post-fit evaluation, and final filtering for Gamma-noise.
<code>03_run_constant_task.R</code>	Corresponding fitting workflow for constant- B .
<code>04_combine_results.R</code>	Validate and combine task-level model outputs.
<code>05_compare_models.R</code>	Verify pairing, calculate recovery summaries, and generate comparison figures.
<code>06_make_convergence_diagnostics.R</code>	Create selected-task IF2 trace diagnostics and tail summaries.
<code>07_generate_selected_B_trajectory.R</code>	Preserve the prespecified task-117 illustrative trajectory artifact.
<code>09_regenerate_sampled_B_trajectories.R</code>	Reconstruct and validate the 200 sampled Gamma paths from saved best-fit parameters.

The following listings restore the implementation detail used to define the models, run the Gamma-noise inference, and calculate the paired recovery summaries. The provenance limitation in Appendix D still applies.

C.3 Experiment configuration

```
1 # =====
2 # Experiment 4 configuration
3 #
4 # Scientific question:
5 # Gamma-noise versus constant-B comparison,
6 # both fitted with Nmif = 600, on exactly the same 200 accepted
7 # simulated epidemic data sets.
8 # =====
9
10 experiment_config <- list(
11   experiment_id = "experiment_4_nmif600_model_comparison",
12   n_tasks = 200L,
13   diagnostic_task_ids = c(1L, 50L, 100L, 150L, 200L),
14
15   n_weeks = 10,
16   observation_interval = 1 / 7,
17   process_delta_t = 1 / 30,
18
19   true_parameters = c(
20     Beta_high = 4,
21     Beta_low = 2,
22     t_switch = 5,
23     mu_IR = 3,
24     N = 10000,
25     rho = 0.5,
26     k = 10
27   ),
28
29   acceptance_threshold = 20,
30   max_simulation_attempts = 10000L,
31
32   Nmif = 600L,
33   Np_mif = 5000L,
34   Np_eval = 50000L,
35   n_pf_evals = 5L,
36   Np_final = 50000L,
37
38   cooling_type = "geometric",
39   cooling_fraction_50 = 0.5,
40
41   gamma_start_values = expand.grid(
42     B0 = c(2, 4, 6),
43     sigma_beta = c(0.10, 0.30, 0.45)
44   ),
45   gamma_rw_sd_B0_ivp = 0.20,
46   gamma_rw_sd_sigma_beta = 0.05,
47
48   constant_start_values = data.frame(
49     Beta = c(1, 2, 3, 4, 5, 6)
50   ),
51   constant_rw_sd_Beta = 0.05
52 )
```

Listing 1: Complete preserved Experiment 4 configuration from config/experiment_config.R, copied as code/experiment4_source/experiment_config.R.

C.4 Data-generating and fitting POMP

```
5 make_observation_template <- function(config) {
6   data.frame(
7     week = seq(
8       from = config$observation_interval,
9       to = config$n_weeks,
10      by = config$observation_interval
11    ),
12    reports = 0
13  )
14 }
15
16 sir_rmeas <- Csnippet("
17   reports = rnbinom_mu(k, rho * H);
18 ")
19
20 sir_dmeas <- Csnippet("
21   lik = dnbinom_mu(
22     reports,
23     k,
24     rho * H,
25     give_log
26   );
27 ")
28
29 make_data_generating_model <- function(config) {
30   template <- make_observation_template(config)
31
32   sir_step_piecewise <- Csnippet("
33     double Beta_now;
34     double dN_SI;
35     double dN_IR;
36     double p_SI;
37     double p_IR;
38
39     if (t < t_switch) {
40       Beta_now = Beta_high;
41     } else {
42       Beta_now = Beta_low;
43     }
44
45     p_SI = 1.0 - exp(-Beta_now * I / N * dt);
46     p_IR = 1.0 - exp(-mu_IR * dt);
47
48     dN_SI = rbinom(S, p_SI);
49     dN_IR = rbinom(I, p_IR);
50
51     S = S - dN_SI;
52     I = I + dN_SI - dN_IR;
53     R = R + dN_IR;
54     H = H + dN_SI;
55   ")
56
57   sir_rinit_piecewise <- Csnippet("
58     S = N - 10;
59     I = 10;
60     R = 0;
61     H = 0;
```

```

62   ")
63
64   pomp(
65     data = template,
66     times = "week",
67     t0 = 0,
68     rinit = sir_rinit_piecewise,
69     rprocess = euler(
70       sir_step_piecewise,
71       delta.t = config$process_delta_t
72     ),
73     rmeasure = sir_rmeas,
74     dmeasure = sir_dmeas,
75     accumvars = "H",
76     statenames = c("S", "I", "R", "H"),
77     paramnames = c(
78       "Beta_high", "Beta_low", "t_switch",
79       "mu_IR", "N", "rho", "k"
80     )
81   )
82 }
83
84 make_gamma_model <- function(observed_data, config) {
85   sir_step_gamma <- Csnippet("
86     double shape_B;
87     double scale_B;
88     double dN_SI;
89     double dN_IR;
90     double p_SI;
91     double p_IR;
92
93     if (sigma_beta > 0.0) {
94       shape_B = 1.0 / (sigma_beta * sigma_beta * dt);
95       scale_B = B * sigma_beta * sigma_beta * dt;
96       B = rgamma(shape_B, scale_B);
97     }
98
99     p_SI = 1.0 - exp(-B * I / N * dt);
100    p_IR = 1.0 - exp(-mu_IR * dt);
101
102    dN_SI = rbinom(S, p_SI);
103    dN_IR = rbinom(I, p_IR);
104
105    S = S - dN_SI;
106    I = I + dN_SI - dN_IR;
107    R = R + dN_IR;
108    H = H + dN_SI;
109  ")
110
111  sir_rinit_gamma <- Csnippet("
112    S = N - 10;
113    I = 10;
114    R = 0;
115    H = 0;
116    B = B0;
117  ")
118
119  pomp(
120    data = observed_data,

```

```

121   times = "week",
122   t0 = 0,
123   rinit = sir_rinit_gamma,
124   rprocess = euler(
125     sir_step_gamma,
126     delta.t = config$process_delta_t
127   ),
128   rmeasure = sir_rmeas,
129   dmeasure = sir_dmeas,
130   accumvars = "H",
131   statenames = c("S", "I", "R", "H", "B"),
132   paramnames = c("B0", "sigma_beta", "mu_IR", "N", "rho", "k"),
133   partrans = parameter_trans(
134     log = c("B0", "sigma_beta")
135   )
136 )
137 }
138
139 make_constant_model <- function(observed_data, config) {
140   sir_step_constant <- Csnippet("
141     double dN_SI;
142     double dN_IR;
143     double p_SI;
144     double p_IR;
145
146     p_SI = 1.0 - exp(-Beta * I / N * dt);
147     p_IR = 1.0 - exp(-mu_IR * dt);
148
149     dN_SI = rbinom(S, p_SI);
150     dN_IR = rbinom(I, p_IR);
151
152     S = S - dN_SI;
153     I = I + dN_SI - dN_IR;
154     R = R + dN_IR;
155     H = H + dN_SI;
156   ")
157
158   sir_rinit_constant <- Csnippet("
159     S = N - 10;
160     I = 10;
161     R = 0;
162     H = 0;
163   ")
164
165   pomp(
166     data = observed_data,
167     times = "week",
168     t0 = 0,
169     rinit = sir_rinit_constant,
170     rprocess = euler(
171       sir_step_constant,
172       delta.t = config$process_delta_t
173     ),
174     rmeasure = sir_rmeas,
175     dmeasure = sir_dmeas,
176     accumvars = "H",
177     statenames = c("S", "I", "R", "H"),
178     paramnames = c("Beta", "mu_IR", "N", "rho", "k"),
179     partrans = parameter_trans(

```

```

180     log = "Beta"
181   )
182 )
183 }

```

Listing 2: Observation template, measurement model, prescribed piecewise data-generating POMP, latent Gamma-noise fitting POMP, and static constant- B fitting POMP from code/model_components.R.

C.5 IF2 fitting and post-fit filtering

```

59 mif_rw_sd <- rw_sd(
60   B0 = ivp(config$gamma_rw_sd_B0_ivp),
61   sigma_beta = config$gamma_rw_sd_sigma_beta
62 )
63 save_traces <- task_id %in% config$diagnostic_task_ids
64
65 cat(
66   "Experiment = ", config$experiment_id,
67   "\nModel = Gamma-noise model",
68   "\nTask ID = ", task_id,
69   "\nObserved-data MD5 = ", shared$observed_data_md5,
70   "\nNmif = ", config$Nmif,
71   "\nNp_mif = ", config$Np_mif,
72   "\nNp_eval = ", config$Np_eval,
73   "\nn_pf_evals = ", config$nn_pf_evals,
74   "\nNp_final = ", config$Np_final,
75   "\nSave convergence traces = ", save_traces,
76   "\n", sep = ""
77 )
78
79 mif_rows <- vector("list", nrow(start_values))
80 eval_rows <- vector("list", nrow(start_values))
81 trace_rows <- vector("list", nrow(start_values))
82
83 for (s in seq_len(nrow(start_values))) {
84   theta_start <- theta_gamma
85   theta_start[["B0"]] <- start_values$B0[[s]]
86   theta_start[["sigma_beta"]] <- start_values$sigma_beta[[s]]
87
88   cat(
89     "Running Gamma-noise model start ", s, "/", nrow(start_values),
90     ": B0=", theta_start[["B0"]],
91     ", sigma_beta=", theta_start[["sigma_beta"]], "\n", sep = ""
92   )
93
94   set.seed(mif_seed_base)
95   mif_now <- tryCatch(
96     mif2(
97       gamma_model,
98       params = theta_start,
99       Np = config$Np_mif,
100       Nmif = config$Nmif,
101       rw.sd = mif_rw_sd,
102       cooling.type = config$cooling_type,
103       cooling.fraction.50 = config$cooling_fraction_50
104     ),

```

Listing 3: Gamma-model parameter perturbations and the corresponding `mif2()` call from `code/02_run_gamma_task.R`.

```

167 eval_logLik <- rep(NA_real_, config$n_pf_evals)
168 eval_status <- rep("failed", config$n_pf_evals)
169
170 for (j in seq_len(config$n_pf_evals)) {
171   set.seed(evaluation_seed_base + j)
172   pf_eval <- tryCatch(
173     pfilter(gamma_model, params = theta_eval, Np = config$Np_eval),
174     error = function(e) NULL
175   )
176   if (!is.null(pf_eval)) {
177     eval_logLik[[j]] <- as.numeric(logLik(pf_eval))
178     if (is.finite(eval_logLik[[j]])) eval_status[[j]] <- "success"
179   }
180 }
181
182 eval_rows[[s]] <- data.frame(
183   run = s,
184   evaluation_rep = seq_len(config$n_pf_evals),
185   logLik = eval_logLik,
186   status = eval_status,
187   stringsAsFactors = FALSE
188 )
189
190 finite_eval <- eval_logLik[is.finite(eval_logLik)]
191 if (length(finite_eval) == 0L) {
192   mif_rows[[s]] <- data.frame(
193     run = s,
194     start_B0 = theta_start[["B0"]],
195     start_sigma_beta = theta_start[["sigma_beta"]],
196     B0_hat = theta_eval[["B0"]],
197     sigma_beta_hat = theta_eval[["sigma_beta"]],
198     Beta_hat = NA_real_,
199     logLik = NA_real_, logLik_se = NA_real_,
200     n_successful_pf_evals = 0L,
201     status = "all_evaluation_pfilters_failed",
202     error_message = NA_character_,
203     stringsAsFactors = FALSE
204   )
205   next
206 }
207
208 likelihood_summary <- pomp::logmeanexp(finite_eval, se = TRUE)
209 mif_rows[[s]] <- data.frame(
210   run = s,
211   start_B0 = theta_start[["B0"]],
212   start_sigma_beta = theta_start[["sigma_beta"]],
213   B0_hat = theta_eval[["B0"]],
214   sigma_beta_hat = theta_eval[["sigma_beta"]],
215   Beta_hat = NA_real_,
216   logLik = as.numeric(likelihood_summary[[1]]),
217   logLik_se = as.numeric(likelihood_summary[[2]]),
218   n_successful_pf_evals = length(finite_eval),
219   status = "success",
220   error_message = NA_character_,

```

```

221     stringsAsFactors = FALSE
222 )
223
224 cat(
225     "Finished start ", s,
226     ": B0_hat=", round(theta_eval[["B0"]], 5),
227     ", sigma_beta_hat=", round(theta_eval[["sigma_beta"]], 5),
228     ", independent logLik=", round(mif_rows[[s]]$logLik[[1]], 4), "\n",
229     sep = ""
230 )

```

Listing 4: Independent post-fit particle-filter evaluations and likelihood-scale combination.

```

257 best <- valid[which.max(valid$logLik), , drop = FALSE]
258 theta_best <- theta_gamma
259 theta_best[["B0"]] <- best$B0_hat[[1]]
260 theta_best[["sigma_beta"]] <- best$sigma_beta_hat[[1]]
261
262 set.seed(final_pf_seed)
263 pf_best <- tryCatch(
264   pfilter(
265     gamma_model,
266     params = theta_best,
267     Np = config$Np_final,
268     filter.traj = TRUE
269   ),
270   error = function(e) structure(
271     list(message = conditionMessage(e)), class = "final_pf_error"
272   )
273 )
274
275 if (inherits(pf_best, "final_pf_error")) {
276   B_path <- empty_path
277   final_pf_logLik <- NA_real_
278   final_pf_success <- FALSE
279   final_status <- "final_pf_failed"
280 } else {
281   sampled <- tryCatch(
282     filter_traj(pf_best, vars = "B", format = "data.frame"),
283     error = function(e) structure(
284       list(message = conditionMessage(e)), class = "trajectory_error"
285     )
286   )
287
288   expected_times <- c(0, observed_data$week)
289   valid_sample <- !inherits(sampled, "trajectory_error") &&
290     all(c("time", "value") %in% names(sampled)) &&
291     nrow(sampled) == length(expected_times) &&
292     all(is.finite(sampled$time)) && all(is.finite(sampled$value)) &&
293     max(abs(sampled$time - expected_times)) <= 1e-12 &&
294     !is.unsorted(sampled$time, strictly = TRUE) &&
295     !anyDuplicated(sampled$time)
296
297   if (!valid_sample) {
298     B_path <- empty_path
299     final_pf_logLik <- as.numeric(logLik(pf_best))
300     final_pf_success <- FALSE
301     final_status <- "sampled_trajectory_extraction_failed"
302   } else {
303     sampled_observation_times <- sampled[-1L, , drop = FALSE]
304     B_path <- data.frame(
305       week = as.numeric(sampled_observation_times$time),
306       B_estimate = as.numeric(sampled_observation_times$value),
307       B_true = true_B_at_times(sampled_observation_times$time, config),
308       trajectory_seed = final_pf_seed,
309       path_semantics = "ancestry_preserving_sampled_latent_trajectory",
310       stringsAsFactors = FALSE
311     )
312     final_pf_logLik <- as.numeric(logLik(pf_best))
313     final_pf_success <- TRUE
314     final_status <- "success"

```

315 }

Listing 5: Best-fit selection, final particle filter with retained ancestry, and sampled-trajectory extraction from code/02_run_gamma_task.R.

C.6 Paired recovery calculations

```
50 pair_check <- merge(  
51   gamma$best[, c("task_id", "simulation_seed", "observed_data_md5", "Nmif")],  
52   constant$best[, c("task_id", "simulation_seed", "observed_data_md5", "Nmif")],  
53   by = "task_id",  
54   suffixes = c("_gamma", "_constant")  
55 )  
56  
57 pair_check$same_simulation_seed <- pair_check$simulation_seed_gamma == pair_check$simulation_seed_  
   constant  
58 pair_check$same_observed_data_md5 <- pair_check$observed_data_md5_gamma == pair_check$observed_data_  
   md5_constant  
59 pair_check$both_nmif_600 <- pair_check$Nmif_gamma == experiment_config$Nmif & pair_check$Nmif_  
   constant == experiment_config$Nmif  
60 write.csv(pair_check, file.path(output_dir, "shared_data_pair_check.csv"), row.names = FALSE)  
61  
62 if (!all(pair_check$same_simulation_seed) || !all(pair_check$same_observed_data_md5) || !all(pair_  
   check$both_nmif_600)) {  
63   stop("The two models did not use identical shared data or identical Nmif=600 settings for every  
     task.")  
64 }  
65  
66 # -----  
67 # Recovery metrics.  
68 # -----  
69  
70 calculate_metrics <- function(paths, model_name) {  
71   required_columns <- c("task_id", "week", "B_estimate", "B_true")  
72   if (!all(required_columns %in% names(paths))) {  
73     stop(model_name, " paths lack required recovery-metric columns.")  
74   }  
75   paths$error <- paths$B_estimate - paths$B_true  
76   paths$squared_error <- paths$error^2  
77   split_paths <- split(paths, paths$task_id)  
78  
79   out <- do.call(rbind, lapply(split_paths, function(x) {  
80     expected_times <- seq(  
81       from = experiment_config$observation_interval,  
82       to = experiment_config$n_weeks,  
83       by = experiment_config$observation_interval  
84     )  
85     if (nrow(x) != 70L || length(expected_times) != 70L ||  
86         max(abs(x$week - expected_times)) > 1e-12 ||  
87         is.unsorted(x$week, strictly = TRUE) || anyDuplicated(x$week)) {  
88       stop(  
89         model_name, " task ", x$task_id[[1]],  
90         " does not contain exactly the 70 configured observation times."  
91       )  
92     }  
93     mean_error <- mean(x$error)  
94     data.frame(  

```



```

95     task_id = x$task_id[[1]],
96     model = model_name,
97     RSS = sum(x$squared_error),
98     RMSE = sqrt(mean(x$squared_error)),
99     mean_error = mean_error,
100    AOB = abs(mean_error),
101    mean_error_before_5 = mean(
102      x$error[x$week < experiment_config$true_parameters[["t_switch"]]]
103    ),
104    mean_error_after_5 = mean(
105      x$error[x$week >= experiment_config$true_parameters[["t_switch"]]]
106    ),
107    stringsAsFactors = FALSE
108  )
109 }))
110 rownames(out) <- NULL
111 out
112 }
113
114 gamma_metrics <- calculate_metrics(gamma$paths, "gamma_noise")
115 constant_metrics <- calculate_metrics(constant$paths, "constant_B")
116 model_task_metrics <- rbind(gamma_metrics, constant_metrics)
117 write.csv(model_task_metrics, file.path(output_dir, "model_task_metrics.csv"), row.names = FALSE)

```

Listing 6: Shared-data validation and task-level recovery calculations from code/05_compare_models.R.

```

167 # -----
168 # Paired task-level comparison table.
169 # -----
170
171 paired <- merge(
172   gamma_metrics,
173   constant_metrics,
174   by = "task_id",
175   suffixes = c("_gamma", "_constant")
176 )
177 paired <- merge(
178   paired,
179   gamma$best[, c("task_id", "logLik", "logLik_se", "B0_hat", "sigma_beta_hat", "status")],
180   by = "task_id"
181 )
182 names(paired)[names(paired) %in% c("logLik", "logLik_se", "B0_hat", "sigma_beta_hat", "status")] <-
183   paste0(
184     names(paired)[names(paired) %in% c("logLik", "logLik_se", "B0_hat", "sigma_beta_hat", "status")],
185     "_gamma"
186   )
187 paired <- merge(
188   paired,
189   constant$best[, c("task_id", "logLik", "logLik_se", "Beta_hat", "status")],
190   by = "task_id",
191   suffixes = c("", "_constant")
192 )
193 names(paired)[names(paired) == "logLik"] <- "logLik_constant"
194 names(paired)[names(paired) == "logLik_se"] <- "logLik_se_constant"
195 names(paired)[names(paired) == "Beta_hat"] <- "Beta_hat_constant"
196 names(paired)[names(paired) == "status"] <- "status_constant"
197
198 paired$delta_RSS_gamma_minus_constant <- paired$RSS_gamma - paired$RSS_constant

```

```

198 paired$delta_RMSE_gamma_minus_constant <- paired$RMSE_gamma - paired$RMSE_constant
199 paired$delta_AOB_gamma_minus_constant <- paired$AOB_gamma - paired$AOB_constant
200 paired$delta_logLik_gamma_minus_constant <- paired$logLik_gamma - paired$logLik_constant
201 paired$gamma_lower_RSS <- paired$RSS_gamma < paired$RSS_constant
202 paired$gamma_lower_RMSE <- paired$RMSE_gamma < paired$RMSE_constant
203 paired$gamma_lower_AOB <- paired$AOB_gamma < paired$AOB_constant
204 write.csv(paired, file.path(output_dir, "paired_model_comparison.csv"), row.names = FALSE)
205
206 overall <- data.frame(
207   quantity = c(
208     "RSS", "RMSE", "AOB (absolute task-level mean error)",
209     "mean error before week 5", "mean error from week 5",
210     "independent log-likelihood", "Gamma-noise model win proportion by RSS",
211     "Gamma-noise model win proportion by RMSE",
212     "Gamma-noise model win proportion by AOB"
213   ),
214   gamma_mean_or_proportion = c(
215     mean(paired$RSS_gamma), mean(paired$RMSE_gamma), mean(paired$AOB_gamma),
216     mean(paired$mean_error_before_5_gamma),
217     mean(paired$mean_error_after_5_gamma), mean(paired$logLik_gamma),
218     mean(paired$gamma_lower_RSS), mean(paired$gamma_lower_RMSE),
219     mean(paired$gamma_lower_AOB)
220   ),
221   constant_mean = c(
222     mean(paired$RSS_constant), mean(paired$RMSE_constant), mean(paired$AOB_constant),
223     mean(paired$mean_error_before_5_constant),
224     mean(paired$mean_error_after_5_constant), mean(paired$logLik_constant),
225     NA_real_, NA_real_, NA_real_
226   ),
227   median_paired_difference_gamma_minus_constant = c(
228     median(paired$delta_RSS_gamma_minus_constant),
229     median(paired$delta_RMSE_gamma_minus_constant),
230     median(paired$delta_AOB_gamma_minus_constant),
231     median(paired$mean_error_before_5_gamma -
232       paired$mean_error_before_5_constant),
233     median(paired$mean_error_after_5_gamma -
234       paired$mean_error_after_5_constant),
235     median(paired$delta_logLik_gamma_minus_constant),
236     NA_real_, NA_real_, NA_real_
237   ),
238   stringsAsFactors = FALSE
239 )
240 write.csv(overall, file.path(output_dir, "overall_model_comparison.csv"), row.names = FALSE)

```

Listing 7: Paired comparison and across-task summaries from code/05_compare_models.R. The implementation field `bias_all` is called task-level mean error in this report.

D Reproducibility information

D.1 Retained inputs and outputs

The repository retains the complete Experiment 4 workflow, the 200 accepted synthetic data sets supplied to both fitting models, combined task-level results, sampled transmission-rate paths, comparison figures, and selected-task convergence diagnostics. The report-specific copy of the R source is stored under `report/code/experiment4_source/`; the canonical executable workflow

is under `experiments/experiment_4_nmif600_model_comparison/code/`. Numerical claims in Section 5 were checked against the retained comparison tables rather than inferred from the figures.

Each shared-data directory records the observed and simulated series, simulation metadata, and file checksums. The manifest at `experiments/experiment_4_nmif600_model_comparison/shared_data/MANIFEST.csv` provides a compact task index. The paired comparison code verifies matching simulation seeds and observed-data checksums before calculating recovery metrics.

D.2 Trajectory and likelihood semantics

Five independent post-IF2 particle filters evaluate each starting-value endpoint. Their likelihood estimates are combined using `pomp::logmeanexp`, and the endpoint with the largest finite summary is selected. This likelihood-scale mean is the only averaging step relevant to best-fit selection; it is unrelated to averaging latent states.

For the Gamma-noise model, a final 50,000-particle filter retains ancestry and `filter_traj()` extracts one sampled latent path. The saved `B_path.csv` values at the 70 observation times are used directly in RSS, RMSE, signed mean error, and absolute overall bias. No filtering mean enters those metrics. The constant-model path is its selected static estimate repeated at the same times.

D.3 Exact-reproduction boundary

The workflow assigns task-specific simulation, IF2, evaluation, and final-filter seeds. Nevertheless, the original high-performance-computing runs did not retain a complete software lockfile. Exact Monte Carlo reproduction can therefore depend on the R, `pomp`, compiler, and random-number-generator versions. Recalculation should preserve the supplied seeds, configuration, and accepted-data checksums, and should report any resulting numerical differences rather than silently replacing the retained canonical outputs.

References

- Judith A. Bouman, Anthony Hauser, Simon L. Grimm, Martin Wohlfender, Samir Bhatt, Elizaveta Semenova, Andrew Gelman, Christian L. Althaus, and Julien Riou. Bayesian workflow for time-varying transmission in stratified compartmental infectious disease transmission models. *PLOS Computational Biology*, 20(4): e1011575, 2024. doi: 10.1371/journal.pcbi.1011575. URL <https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1011575>.
- Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, 2009. doi: 10.1214/08-AOAS201.
- N. J. Gordon, D. J. Salmond, and A. F. M. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F: Radar and Signal Processing*, 140(2):107–113, 1993. doi: 10.1049/ip-f-2.1993.0015.
- Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, 2015. doi: 10.1073/pnas.1410597112.
- Aaron A. King, Dao Nguyen, and Edward L. Ionides. Statistical inference for partially observed markov processes via the R package pomp. *Journal of Statistical Software*, 69(12):1–43, 2016. doi: 10.18637/jss.v069.i12.
- Aaron A. King, Edward L. Ionides, and Carles Bretó. *pomp: Statistical Inference for Partially Observed Markov Processes*. Comprehensive R Archive Network, 2025. URL <https://CRAN.R-project.org/package=pomp>. R package version 6.4.